

vCenter 跨站搬移工具

資料夾 / 自訂屬性 / Notes / Tag 與 VM 註冊

使用手冊與實測報告

Repo : kostenyang/vc-folder-tag-migrate (private)

版本 1.0 2026-09-14

PowerCLI 13.5 / PowerShell 7 實測環境 : vCenter 8.0.3 → vCenter 9.1.1

目錄

1. 概述

1.1 腳本一覽

2. 前置需求

3. 流程總覽：五個動作

3.1 為什麼是這個順序

3.2 完整指令 (把 <...> 換成你的)

4. 各動作詳解

4.0 動 0：Export-VcMeta.ps1 (舊 vC)

4.1 動 1：Import-VcMeta.ps1 -Include Folders (新 vC)

4.2 動 2：Import-VcMeta.ps1 -Include CustomAttributes,Tags (新 vC)

4.3 動 3：Unregister-VmFromOldVc.ps1 (舊 vC)

4.4 動 4：Register-VmxFromDatastore.ps1 (新 vC)

4.5 動 5：Import-VcMeta.ps1 -Include CustomAttributes,Notes,Tags (新 vC)

5. 其他情境

5.1 只搬一個資料夾 (by folder)

5.2 VM 已經在新 vC，只補中繼資料

5.3 人工編修 CSV

6. 比對規則、報告與限制

6.1 比對規則

6.2 報告動作字典

6.3 已知限制

7. 實測報告 (2026-09-14)

7.1 環境

7.2 動 0：舊 vC 匯出

7.3 動 1：新 vC 建資料夾

7.4 動 2：新 vC 建屬性定義與 tag 分類

7.5 動 3：舊 vC unregister

7.6 動 4：新 vC 依新 CSV 註冊

7.7 動 5：新 vC 補屬性值 / Notes / tag

7.8 核對結果

7.9 冪等性與分批

7.10 自我測試

8. 附錄：實作過程抓到的坑

1. 概述

把 VM 從舊 vCenter (A) 搬到新 vCenter (B) 時，vmx / vmdk 跟著 datastore 走，但 inventory 上的東西不會：VM 在哪個資料夾、自訂屬性 (Custom Attributes) 的值、Notes、Tag 指派——這些存在 vCenter 資料庫，VM 一 unregister 就沒了。這套工具把它們落地成 CSV，等 VM 在新 vC 註冊回來後再原樣補上。

設計上的三個原則：

- **一次一動、每動可獨立驗證與重跑。**五個動作各一支腳本，全部 idempotent，中間隔多久都可以，兩台 vC 不必同時在線。
- **先 DryRun 再做。**每支腳本都有 -DryRun，只比對、不寫入，報告告訴你「本來會做什麼」。
- **落地檔就是清單。**VM 兩邊都看不到那段時間，匯出目錄裡的 vm-placement.csv 與 unregistered.csv 就是真相源，新 vC 端會拿它對帳。

1.1 腳本一覽

腳本	在哪跑	用途
Export-VcMeta.ps1	舊 vC	動 0：匯出資料夾樹、VM 位置、tag、自訂屬性、Notes 成 8 個 CSV
Import-VcMeta.ps1	新 vC	動 1 / 2 / 5：把 CSV 匯入 (-Include 選要做哪幾類)
Unregister-VmFromOldVc.ps1	舊 vC	動 3：依清單把 VM 移出 inventory (檔案留著)，寫 unregistered.csv
Register-VmxFromDatastore.ps1	新 vC	動 4：依動 3 產生的 unregistered.csv 逐台註冊回來、直接放進原資料夾、對帳 (也可掃整顆 datastore)
Copy-VcMeta.ps1 / Copy-VcCustomAttributes.ps1 / Copy-VcFolders.ps1	任一台	兩台 vC 同時連得到時的一支到底版 (內部就是 Export → Import)
Test-VcMeta.ps1 / Test-RegisterVmx.ps1	—	端到端自我測試：建測試物件 → 跑 → 獨立驗證 → 清除

2. 前置需求

項目	需求
執行環境	PowerShell 7 (pwsh) + VMware.PowerCLI 13.x (實測 13.5.0)。Windows PowerShell 5.1 也可，但 CSV 編碼參數會自動切換。
帳號	兩台 vC 都要能建資料夾、建 / 指派 tag、建自訂屬性、註冊 / 移除 VM 的權限；實測用 administrator@vsphere.local。
網路	執行機要能連到 vC 的 443。腳本已設 InvalidCertificateAction Ignore，自簽憑證不用先信任。

項目	需求
儲存	動 4 之前，VM 所在的 datastore 要已經掛在新 vC 的至少一台主機上（共用 NFS / SAN LUN 重新 present 皆可）。
Datacenter 名稱	兩邊不同時用 -DatacenterMap '舊 DC=新 DC'（可多組）。

注意 多值參數（-Include、-DatacenterMap、-Folder、-Datastore）一定要用 `powershell -Command "& .\腳本.ps1 ..." 呼叫`；用 `powershell -File` 會把逗號清單當成一個字串，參數驗證直接失敗。本文所有指令都用 -Command 寫法。

3. 流程總覽：五個動作

動	在哪	腳本	做什麼
0	舊 vC	Export-VcMeta	一次落地全部：資料夾 / VM 位置 / tag / 屬性 / Notes → export-A\
1	新 vC	Import-VcMeta -Include Folders	資料夾樹先建好
2	新 vC	Import-VcMeta -Include CustomAttributes,Tags	屬性定義、tag 分類 / 標籤先建好（VM 還沒過去，值和指派先不會有）
3	舊 vC	Unregister-VmFromOldVc	VM 移出舊 vC，產生新 CSV export-A\unregistered.csv（誰 / vmx / 原資料夾）
—	儲存	（人工）	datastore 卸載、搬到新 vC、掛上主機。隔多久都可以
4	新 vC	Register-VmxFromDatastore -UnregisteredCsv	依新 CSV 逐台註冊、直接放進原資料夾；結尾對帳：誰還沒過來
5	新 vC	Import-VcMeta -Include CustomAttributes,Notes,Tags	補 VM 的屬性值 / Notes / tag 指派（VM 已在，全部對得到）

3.1 為什麼是這個順序

- 動 0 必須在動 3 之前。VM — unregister，舊 vC 上它的 tag 指派、屬性值就消失了；匯出檔是唯一的來源。動 3 的腳本沒有匯出檔不給做。
- 動 1、2 可以提早做。資料夾、屬性定義、tag 分類都是 vCenter 層物件，跟 VM 無關，VM 過去之前就能備妥；動 4 註冊時資料夾已經在。
- 動 2 和動 5 是同一支腳本跑兩次。第一次建定義（值對不到 VM 是預期），第二次補值和指派。這是 vCenter 的限制——值要掛在物件上——不是腳本的。
- 動 4 的輸入是動 3 產生的新 CSV。unregistered.csv 精確記錄拔掉的那批（vmx 路徑 / 名稱 / 原資料夾），註冊就照它做、不去猜；可以先用 Excel 改再跑。可以分批，每次結尾都對帳。

3.2 完整指令 (把 <...> 換成你的)

```
# 動 0 ( 舊 vC )
pwsh -Command "& .\Export-VcMeta.ps1 -Server <舊 vC> -User administrator@vsphere.local -Password '<pw>' -OutDir .\export-A -Folder 'Linux'"

# 動 1 ( 新 vC )
pwsh -Command "& .\Import-VcMeta.ps1 -Server <新 vC> -User administrator@vsphere.local -Password '<pw>' -InDir .\export-A -DatacenterMap '<舊 DC>=<新 DC>' -Include Folders"

# 動 2 ( 新 vC )
pwsh -Command "& .\Import-VcMeta.ps1 -Server <新 vC> -User administrator@vsphere.local -Password '<pw>' -InDir .\export-A -DatacenterMap '<舊 DC>=<新 DC>' -Include CustomAttributes,Tags"

# 動 3 ( 舊 vC )
pwsh -Command "& .\Unregister-VmFromOldVc.ps1 -Server <舊 vC> -Password '<pw>' -MetaDir .\export-A -Cluster cl01 -Datastore ds01"

# 動 4 ( 新 vC )
pwsh -Command "& .\Register-VmxFromDatastore.ps1 -Server <新 vC> -User administrator@vsphere.local -Password '<pw>' -Cluster cl01 -UnregisteredCsv .\export-A\unregistered.csv"

# 動 5 ( 新 vC )
pwsh -Command "& .\Import-VcMeta.ps1 -Server <新 vC> -User administrator@vsphere.local -Password '<pw>' -InDir .\export-A -DatacenterMap '<舊 DC>=<新 DC>' -Include CustomAttributes,Notes,Tags"
```

每一條都可以先加 -DryRun 看報告，確認後拿掉再跑。

4. 各動作詳解

4.0 動 0 : Export-VcMeta.ps1 (舊 vC)

把舊 vC 的 inventory 中繼資料落地成 8 個 CSV (UTF-8 with BOM，Excel 直接開、可人工編修後再匯入)。

參數	說明
-Server / -User / -Password	舊 vC 連線資訊 (也可用 -Credential)
-OutDir	輸出目錄，預設 .\vc-meta-export
-Folder 'Linux','MGMT/Prod'	只匯出這些 VM 資料夾子樹與裡面的 VM；tag / 屬性定義只帶「範圍內物件真的用到」的 (-AllDefinitions 可全帶)
-Datacenter	只匯出指定 Datacenter (可多個)
-Include	Folders, VMPlacement, Tags, CustomAttributes, Notes (預設全要)

檔案	內容
folders.csv	資料夾樹：Datacenter / 類型 (VM、HostAndCluster、Datastore、Network) / 相對路徑 / 深度
vm-placement.csv	每台 VM / 範本在哪個資料夾；含 InstanceUuid、BiosUuid、

檔案	內容
	IsTemplate、InVApp、VmPathName、PowerState、VMHost、Cluster
tag-categories.csv / tags.csv	Tag 分類 (Cardinality、EntityType) 與標籤
tag-assignments.csv	誰被貼了什麼標籤 (VM 帶 UUID、資料夾帶路徑)
custom-attributes.csv / custom-attribute-values.csv	自訂屬性定義 (含 TargetType) 與每個物件的值
notes.csv	VM Notes

注意 vm-placement.csv 的 VmPathName / PowerState 兩欄是動 3 的安全檢查與動 4 對帳的依據，請不要手動刪掉這兩欄。

4.1 動 1 : Import-VcMeta.ps1 -Include Folders (新 vC)

依 folders.csv 在新 vC 重建資料夾樹 (由淺到深)，已存在的跳過。用 -DatacenterMap 對應兩邊不同的 Datacenter 名稱。

參數	說明
-InDir	動 0 的輸出目錄
-DatacenterMap '舊 DC=新 DC'	兩邊 Datacenter 名稱不同時的對應，可多組
-Include	要做哪幾類；動 1 只給 Folders
-Folder 'Linux'	用整台 vC 的匯出檔也能只匯入某個資料夾子樹
-ReportPath	明細報告，預設寫在 -InDir 下 import-report- <時間> .csv

4.2 動 2 : Import-VcMeta.ps1 -Include CustomAttributes,Tags (新 vC)

建自訂屬性定義 (含 TargetType)、tag 分類 (Cardinality / EntityType) 與標籤。這時 VM 還沒在新 vC，報告會出現 CAValue/EntityNotFound、TagAssignment/EntityNotFound——這是預期，動 5 會補。

注意 目標端已有同名 tag 分類時沿用既有設定 (不改 Cardinality / EntityType)；同名自訂屬性視為已存在。

4.3 動 3 : Unregister-VmFromOldVc.ps1 (舊 vC)

依匯出清單把 VM / 範本從舊 vC 的 inventory 移除，檔案留在 datastore。這一步做完，VM 在兩邊都看不到，直到動 4——所以腳本內建幾道安全機制。

參數	說明
-MetaDir	動 0 的輸出目錄 (必填；沒有 vm-placement.csv 直接拒絕)
-Datastore ds01	清單裡 vmx 在這些 datastore 上的 VM (= VM 的 home datastore)

參數	說明
-Lun naa.6000... / 尾碼	儲存端給的 LUN ID：用 VMFS extent 反查落在上面的 datastore，效果同 -Datastore
-Folder 'Linux'	清單裡在這些資料夾子樹的 VM
-VM web01,web02	直接點名
-ShutdownFirst / -ShutdownTimeoutSec	開著的 VM 先 guest shutdown (需 VMware Tools)，預設不動
-DryRun	只檢查、不 unregister

安全機制：

- 沒有匯出檔不給做——unregister 之後舊 vC 上的 tag 指派、屬性值就沒了。
- 清單裡的 vmx 路徑要跟現在一致 (SkippedStale)，防止拿舊的匯出檔來操作。
- 開機中的 VM 跳過 (SkippedPoweredOn)，除非加 -ShutdownFirst。
- 同名多台跳過 (SkippedAmbiguous)；不接受「全部」，一定要給 -Datastore / -Lun / -Cluster / -VMHost / -Folder / -VM 其中一個。
- 跨 datastore 的 VM 只警告不擋：名單內有 vmdk 在別顆 (WarnDiskElsewhere)、名單外卻有 vmdk 在這顆 (WarnDiskOnDatastore)——只搬一顆 LUN 這兩種都會壞，先把碟搬齊或一起搬。
- 每台成功 unregister 的都追加到 <MetaDir>\unregistered.csv (名稱 / vmx / 時間 / 原資料夾 / UUID)，這就是動 4 對帳的依據。

4.4 動 4：Register-VmxFromDatastore.ps1 (新 vC)

主線用 -UnregisteredCsv：依動 3 產生的 unregistered.csv 逐台註冊，vmx 路徑、名稱、是否範本、原資料夾都從 CSV 來，註冊時直接放進原資料夾 (資料夾由動 1 建好，缺的會自動建)。只註冊、不開機；.vmtx 註冊成範本；已註冊的依「[datastore] 路徑」比對一律跳過，重跑安全。

參數	說明
-UnregisteredCsv	動 3 產生的 export-A\unregistered.csv (主線)。可先用 Excel 改 FolderPath 等欄位再跑
-Cluster / -VMHost	註冊到哪：叢集內有掛該 datastore 的主機輪流用，或指定單一主機
-ResourcePool	預設用主機所屬叢集 / 主機的根 resource pool
-Datastore ds01 -PlacementCsv ...	另一種模式：掃整顆 datastore 的 .vmx / .vmtx，依 vm-placement.csv 放資料夾 (沒有 CSV、清孤兒時用)
-Include / -Exclude	名稱 wildcard；預設排除 vCLS*
-NoTemplates	略過 .vmtx
-MetaDir / -DatacenterMap	(可選) 把動 5 併進來一次做；主線不用
-DryRun	只檢查、不註冊

對帳：結尾拿 unregistered.csv 比對新 vC，列出 PendingOnSource——舊 vC 拔掉了、但新 vC 還沒有的。分批跑每次都會報，DryRun 會加註「這次會註冊幾台、跑完剩幾台」。

注意 開機時 vSphere 若詢問「moved / copied」，回答 I moved it；腳本不代答，也不開機。

4.5 動 5：Import-VcMeta.ps1 -Include CustomAttributes,Notes,Tags (新 vC)

VM 已經在新 vC 了，這一步把屬性值、Notes、tag 指派補上。VM 比對先用 InstanceUuid (實測跨 vC 註冊後不變)，找不到再用名稱。Notes 其實存在 vmx 裡，註冊就自帶，報告會顯示 Notes/AlreadySet。

5. 其他情境

5.1 只搬一個資料夾 (by folder)

動 0 加 -Folder 縮範圍，後面每一動照做；或者動 0 匯整台，之後每一動加 -Folder 分批。範圍規則：

項目	-Folder 下的行為
資料夾	指定路徑本身 + 所有子資料夾 (VM 類型)
VM / 範本	只有這棵子樹裡的
Notes	只有範圍內 VM 的
Tag 指派 / 屬性值	只有範圍內的資料夾與 VM；host / datastore / cluster 一律不帶
分類、標籤、屬性定義	只帶範圍內物件真的用到的 (-AllDefinitions 可全帶)

5.2 VM 已經在新 vC，只補中繼資料

兩台 vC 同時連得到時，用一支到底的版本 (內部就是 Export → Import，CSV 留在 -WorkDir)：

```
pwsh -Command "& .\Copy-VcCustomAttributes.ps1 -SourceServer <舊 vC> -SourcePassword '<pw>'
-TargetServer <新 vC> -TargetPassword '<pw>' -DatacenterMap '<舊 DC>=<新 DC>' -DryRun"
pwsh -Command "& .\Copy-VcFolders.ps1 ... -MoveVMs -DryRun"          # 新 vC 已有同名 / 同 UUID 的 VM 就順便搬進資料夾
pwsh -Command "& .\Copy-VcMeta.ps1 ... -Include Tags,Notes"          # 自選組合
```

5.3 人工編修 CSV

所有 CSV 都可以在 Excel 改了再匯入：改資料夾路徑、刪掉不要搬的 VM、手動加一列 Notes (UUID 留空就走名稱比對)。自我測試有專門驗這條路。

6. 比對規則、報告與限制

6.1 比對規則

物件	比對方式
VM / 範本	先 InstanceUuid，找不到再用名稱；名稱重複則跳過並記錄
Host / Datastore / Cluster / Resource Pool / DS Cluster / DPortgroup	名稱
資料夾	Datacenter + 類型 + 相對路徑（不是只看名稱）
Datacenter	名稱，可用 -DatacenterMap 對應
已註冊的 vmx	[datastore] 相對路徑（不分大小寫）

6.2 報告動作字典

每次執行都產一份明細 CSV (Section / Action / Target / Detail)，畫面上的「結果」是各動作的計數。

動作	意思
Created / Set / Assigned / Moved / Registered / Placed / Unregistered	真的做了
Would*	DryRun 模式下「本來會做」
Exists / AlreadySet / AlreadyThere / AlreadyRegistered	目標端已經一樣，跳過（重跑不會壞）
VMNotFound / EntityNotFound / FolderNotFound	目標端找不到對應物件——通常是 VM 還沒過去，過去後再跑一次就會補上
Skipped* (PoweredOn / Stale / Ambiguous / NotFound)	動 3 的安全機制擋下來，Detail 說明原因
PendingOnSource	動 4 對帳：舊 vC 拔掉了、新 vC 還沒有
Failed	真的失敗，Detail 有錯誤訊息

6.3 已知限制

- vApp 內的 VM (InVApp=True) 不會被搬進資料夾；它的 tag / 屬性 / Notes 照樣會套。
- Notes 是覆蓋不是附加；目標端已有相同內容則跳過。
- 只搬 inventory 中繼資料，不搬權限 (Permission)、Role、Alarm、Resource Pool 結構、DRS / HA 規則。
- 來源若有無法存取的 datastore，Get-TagAssignment 一次撈會中斷——腳本已改成分批、失敗再逐一 fallback。

- vCenter 的 tagging 服務 (vAPI) 偶爾在 session 剛建立時回 503——tag 相關呼叫已包 3 次退避重試。

7. 實測報告 (2026-09-14)

7.1 環境

項目	內容
舊 vC (A)	10.0.0.101 · vCenter 8.0.3 build 24022515 · Datacenter 「Datacenter」 · 主機 10.0.0.95
新 vC (B)	10.0.1.19 · vCenter 9.1.1 build 25712839 · Datacenter 「m01-dc01」 · 主機 vcd-esx01.home.lab
共用儲存	NFS 10.0.0.60:/nfs/vc-migtest · 掛成 datastore 「vc-migtest」到兩邊各一台主機 (191 GB free 兩邊一致)
執行機	Windows Server 2022 · pwsh 7 · PowerCLI 13.5.0
測試資料	兩台 VM : zz-real-vm01 (關機) 、zz-real-vm02 (開機 · 用來驗證動 3 的擋下機制) ；各有 1 GB vmdk 。\$7.6 另一輪加一個範本
畫面截圖	以 Chrome DevTools Protocol 自動化擷取 vSphere Client 畫面 (同一次實跑 · 非事後補拍)

兩台 VM 在舊 vC 的初始狀態 (皆在資料夾 ZZ-Real/Sub · 自訂屬性 zz-real-owner 、Notes 、tag zz-real-env/prod) ：

Name	PowerState	Folder	Vmx
zz-real-vm02	PoweredOn	ZZ-Real/Sub	[vc-migtest] zz-real-vm02/zz-real-vm02.vmx
zz-real-vm01	PoweredOff	ZZ-Real/Sub	[vc-migtest] zz-real-vm01/zz-real-vm01.vmx

vSphere Client 上的搬前狀態 (舊 vC) ：

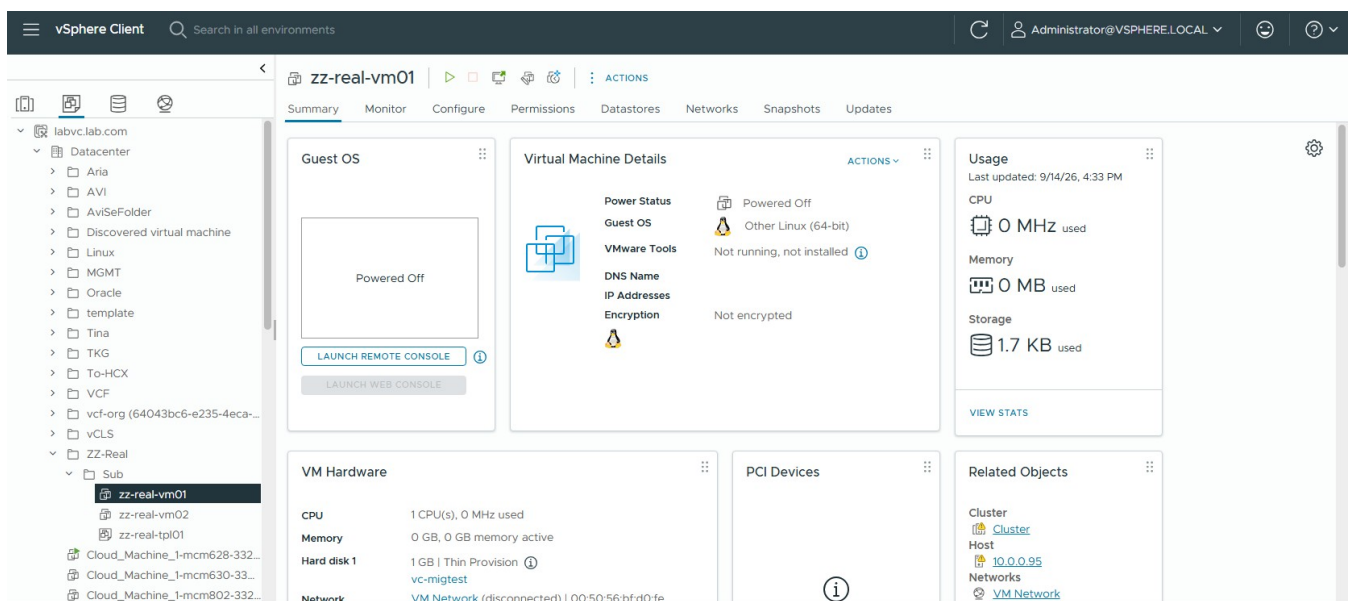


圖 1 舊 vC : 資料夾樹 ZZ-Real/Sub 下有 zz-real-vm01 、vm02 與範本 tpl01

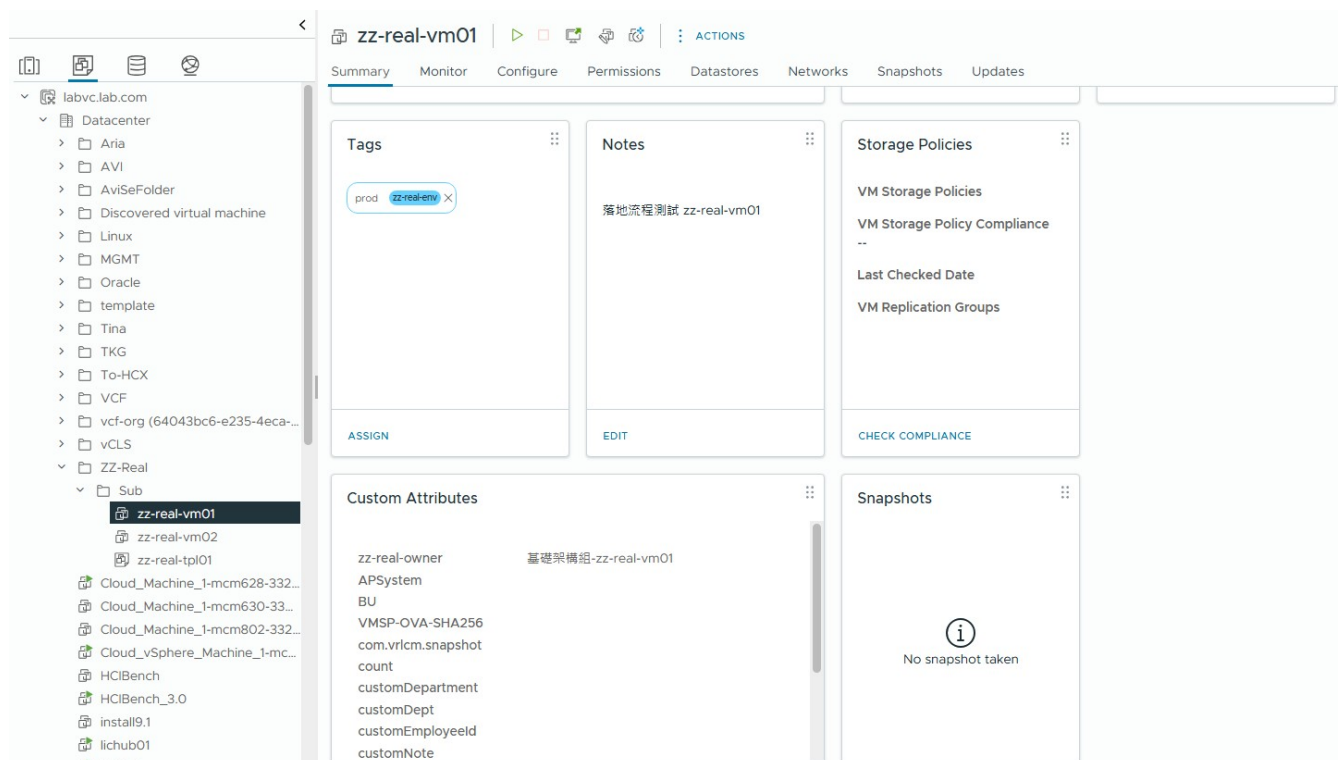


圖 2 舊 vC : zz-real-vm01 的 Tags (prod / zz-real-env) 、 Notes 、 Custom Attributes (zz-real-owner)

7.2 動 0 : 舊 vC 匯出

```
> Export-VcMeta.ps1 -Server 10.0.0.101 -OutDir export-A -Folder ZZ-Real
[+] 已連線 10.0.0.101 (8.0.3 build 24022515)
[*] 範圍限定 : ZZ-Real → 2 個資料夾(含子樹)
-> folders.csv 2 筆
-> vm-placement.csv 2 筆
-> notes.csv 2 筆
-> tag-categories.csv 1 筆
-> tags.csv 1 筆
-> tag-assignments.csv 2 筆
-> custom-attributes.csv 1 筆
-> custom-attribute-values.csv 2 筆
[+] 匯出完成 -> export-A
```

vm-placement.csv 內容 (含 vmx 路徑與電源狀態) :

```
"VMName","InstanceUuid","FolderPath","VmPathName","PowerState"
"zz-real-vm01","503f0e10-ee95-2496-4e63-5372b4a22aa6","ZZ-Real/Sub","[vc-migtest] zz-real-vm01/zz-real-vm01.vmx","poweredOff"
"zz-real-vm02","503f4f6e-1f0a-ee9a-5e4a-36cd766d8635","ZZ-Real/Sub","[vc-migtest] zz-real-vm02/zz-real-vm02.vmx","poweredOn"
```

7.3 動 1 : 新 vC 建資料夾

```
> Import-VcMeta.ps1 -Server 10.0.1.19 -InDir export-A -DatacenterMap Datacenter=m01-dc01 -Include Folders
[+] 已連線 10.0.1.19 (9.1.1 build 25712839)
[1] Folder 結構
===== 結果 =====
Folder/Created 2
```

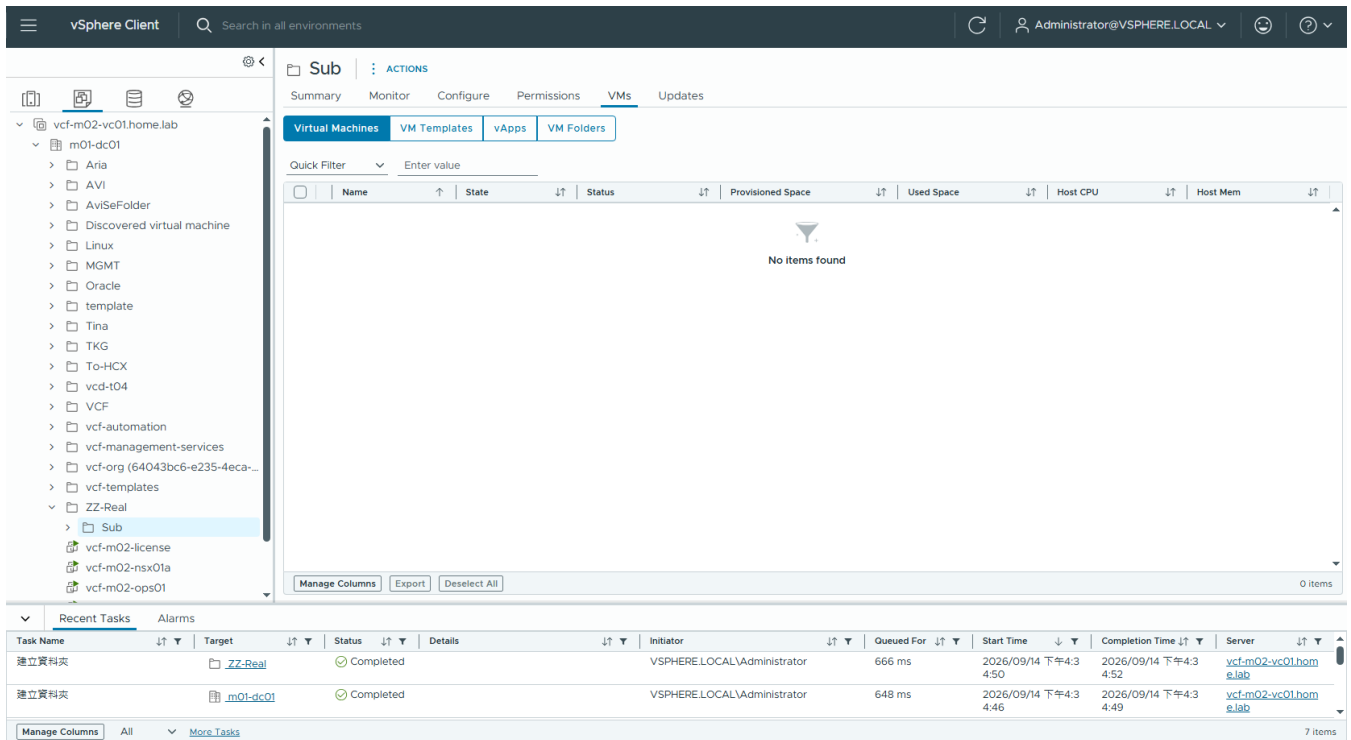


圖 3 新 vC：動 1 之後 ZZ-Real/Sub 已建好但還是空的；Recent Tasks 可見腳本建立資料夾的工作

7.4 動 2：新 vC 建屬性定義與 tag 分類

```
> Import-VcMeta.ps1 ... -Include CustomAttributes,Tags
[2] Custom Attributes
[3] Tags
===== 結果 =====
CAValue/EntityNotFound          2    ← VM 還沒過去，預期
CustomAttribute/Created         1
Tag/Created                     1
TagAssignment/EntityNotFound     2    ← VM 還沒過去，預期
TagCategory/Created             1
```

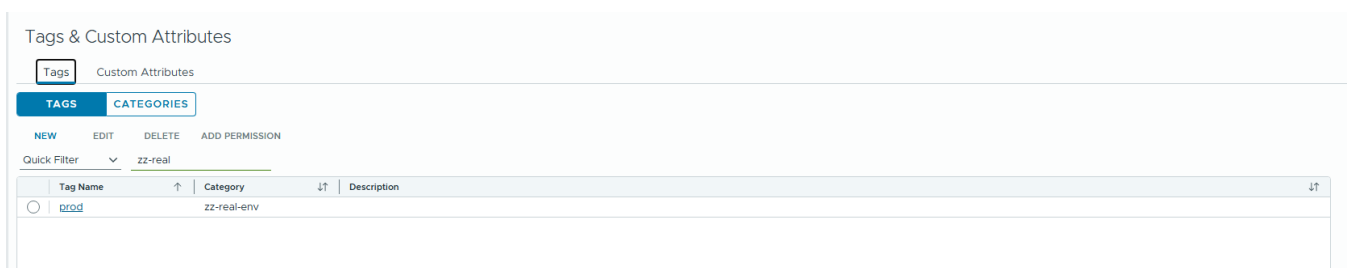


圖 4 新 vC：Tags & Custom Attributes → Tags，篩 zz-real 只有 prod / zz-real-env (動 2 建的)

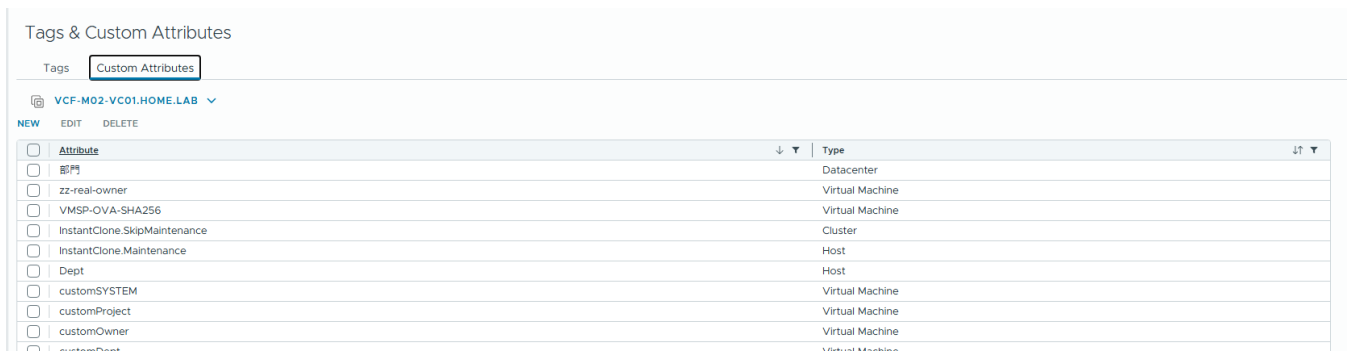


圖 5 新 vC : Custom Attributes 清單已有 zz-real-owner (Virtual Machine 型)

7.5 動 3 : 舊 vC unregister

先 DryRun——vm02 當時是開著的，被正確擋下：

```
> Unregister-VmFromOldVc.ps1 -Server 10.0.0.101 -MetaDir export-A -Datastore vc-migtest -DryRun
[*] 清單 2 台，符合條件 2 台
[!] DryRun : 只檢查、不 unregister
[~] WouldUnregister      zz-real-vm01 [vc-migtest] zz-real-vm01/zz-real-vm01.vmx
[-] SkippedPoweredOn    zz-real-vm02 PoweredOn · 先關機或加 -ShutdownFirst
```

手動關掉 vm02 後正式跑：

```
> Unregister-VmFromOldVc.ps1 -Server 10.0.0.101 -MetaDir export-A -Datastore vc-migtest
[+] Unregistered      zz-real-vm01 [vc-migtest] zz-real-vm01/zz-real-vm01.vmx
[+] Unregistered      zz-real-vm02 [vc-migtest] zz-real-vm02/zz-real-vm02.vmx
[+] 已 unregister 的 2 台記在 export-A\unregistered.csv ( 新 vC 端對帳用 )
```

export-A\unregistered.csv：

```
"Time","VMName","IsTemplate","VmPathName","FolderPath","InstanceUuid","Server"
"2026-09-14 15:42:46","zz-real-vm01","False","[vc-migtest]
zz-real-vm01/zz-real-vm01.vmx","ZZ-Real/Sub","503f0e10-...", "10.0.0.101"
"2026-09-14 15:42:47","zz-real-vm02","False","[vc-migtest]
zz-real-vm02/zz-real-vm02.vmx","ZZ-Real/Sub","503f4f6e-...", "10.0.0.101"
```

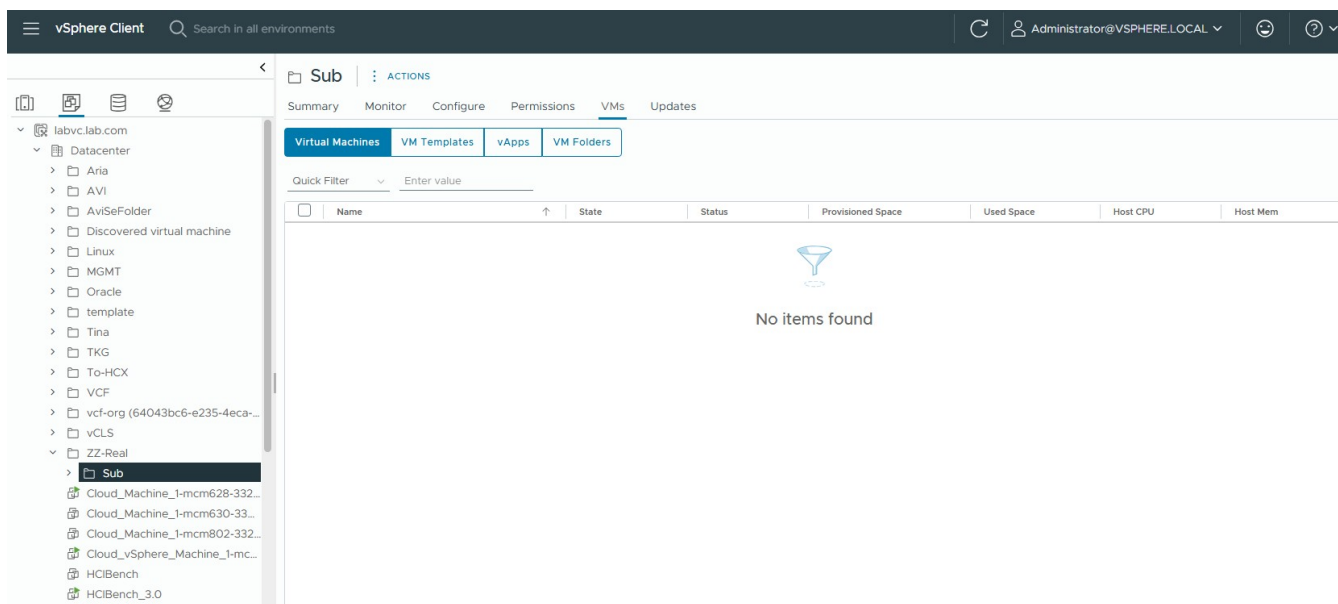


圖 6 舊 vC：動 3 之後 ZZ-Real/Sub 已空 (VM 移出 inventory，檔案還在 datastore)

7.6 動 4：新 vC 依新 CSV 註冊

另一輪實測 (同環境，多加一個範本 zz-real-tpl01)：動 3 用 -Cluster Cluster -Datastore vc-migtest 交集篩出 3 台並產生 unregistered.csv，動 4 直接吃它：

```
> Register-VmxFromDatastore.ps1 -Server 10.0.1.19 -VMHost vcd-esx01.home.lab -UnregisteredCsv
export-B\unregistered.csv -DryRun
=== 依 unregistered.csv 註冊 3 台 ===
datastore vc-migtest -> 主機: vcd-esx01.home.lab
[~] WouldRegister [vc-migtest] zz-real-tpl01/zz-real-tpl01.vmtx Template 'zz-real-tpl01' ->
host=vcd-esx01.home.lab folder=ZZ-Real/Sub
[~] WouldRegister [vc-migtest] zz-real-vm01/zz-real-vm01.vmx VM 'zz-real-vm01' -> host=vcd-
esx01.home.lab folder=ZZ-Real/Sub
[~] WouldRegister [vc-migtest] zz-real-vm02/zz-real-vm02.vmx VM 'zz-real-vm02' -> host=vcd-
esx01.home.lab folder=ZZ-Real/Sub
===== 對帳 (unregistered.csv vs 目標端) =====
舊 vC 端位於 vc-migtest 的 VM: 3 台; 目標端已有 0 台; 還沒過來 3 台 ( 這次會註冊 3 台, 跑完剩 0 台 )
```

正式跑：

```
> Register-VmxFromDatastore.ps1 ... -UnregisteredCsv export-B\unregistered.csv
[+] Registered [vc-migtest] zz-real-tpl01/zz-real-tpl01.vmtx Template 'zz-real-tpl01' on vcd-
esx01.home.lab -> ZZ-Real/Sub
[+] Registered [vc-migtest] zz-real-vm01/zz-real-vm01.vmx VM 'zz-real-vm01' on vcd-
esx01.home.lab -> ZZ-Real/Sub
[+] Registered [vc-migtest] zz-real-vm02/zz-real-vm02.vmx VM 'zz-real-vm02' on vcd-
esx01.home.lab -> ZZ-Real/Sub
===== 對帳 (unregistered.csv vs 目標端) =====
舊 vC 端位於 vc-migtest 的 VM: 3 台; 目標端已有 3 台; 還沒過來 0 台
```

新 vC 獨立查詢：三個物件都在 ZZ-Real/Sub，範本仍是範本 (Type=Template)，兩台 VM 動 5 之後屬性與 tag 到位；舊 vC 已無 zz-real-*。

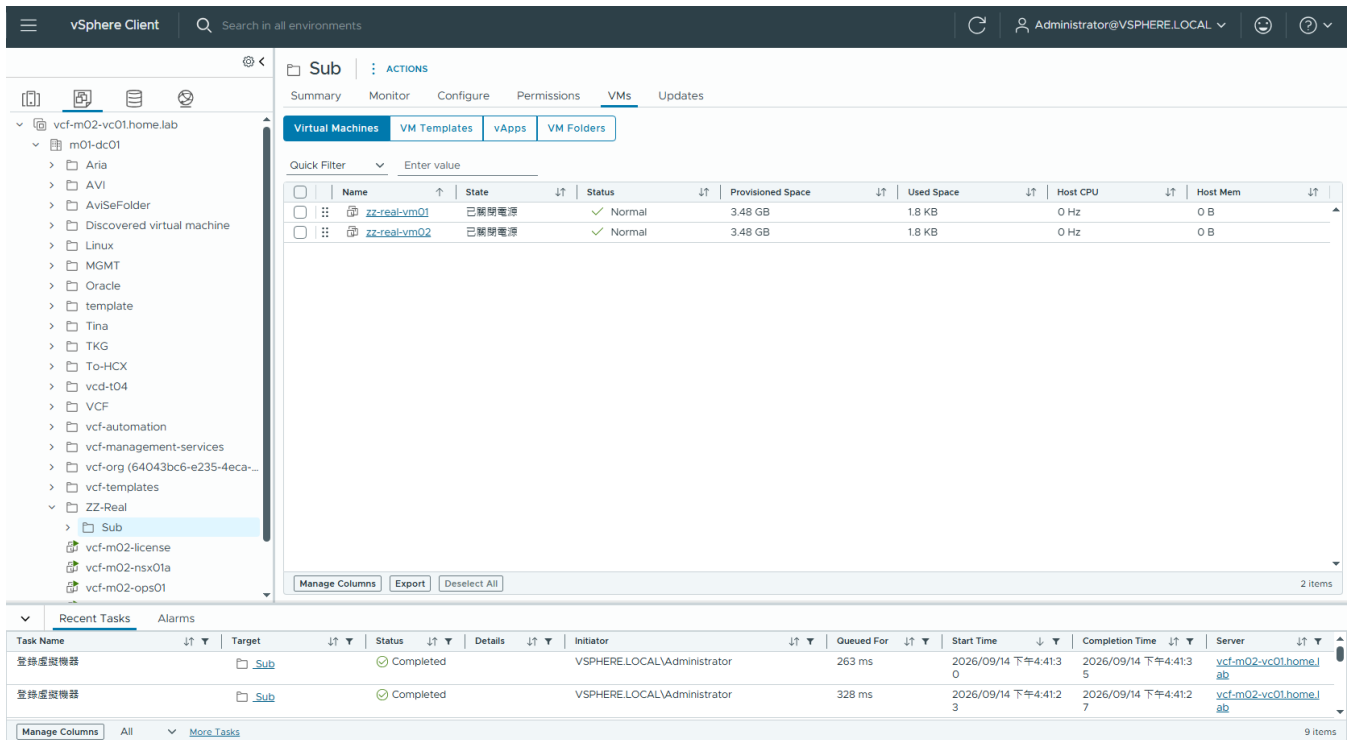


圖 7 新 vC：動 4 之後 ZZ-Real/Sub 的 VMs 頁有 vm01、vm02（範本在 VM Templates 頁）；Recent Tasks 為「登錄虛擬機器」

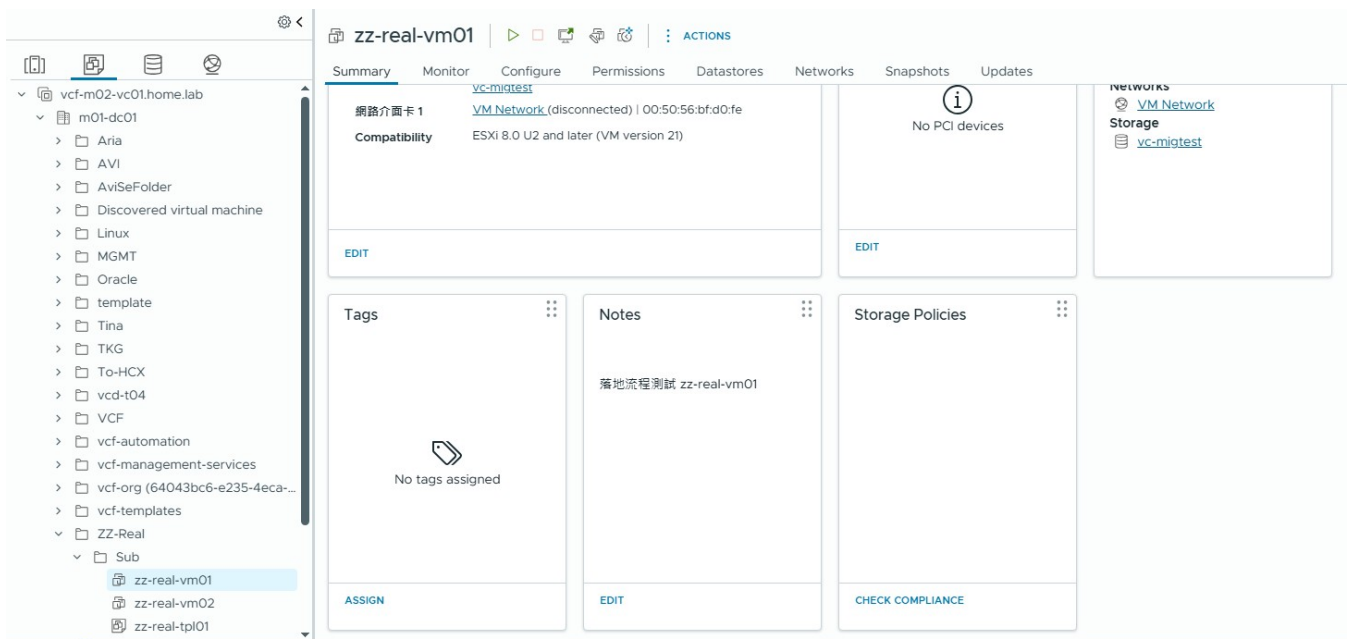


圖 8 新 vC：動 5 之前的 zz-real-vm01——Notes 已在（vmx 自帶）· Tags 是 No tags assigned

7.7 動 5：新 vC 補屬性值 / Notes / tag

```
> Import-VcMeta.ps1 ... -Include CustomAttributes,Notes,Tags
[2] Custom Attributes
[3] Tags
[4] VM Notes
===== 結果 =====
CAValue/Set                2
CustomAttribute/Exists     1
Notes/AlreadySet           2    ← Notes 存在 vmx 裡，註冊就自帶
Tag/Exists                  1
TagAssignment/Assigned     2
TagCategory/Exists         1
```

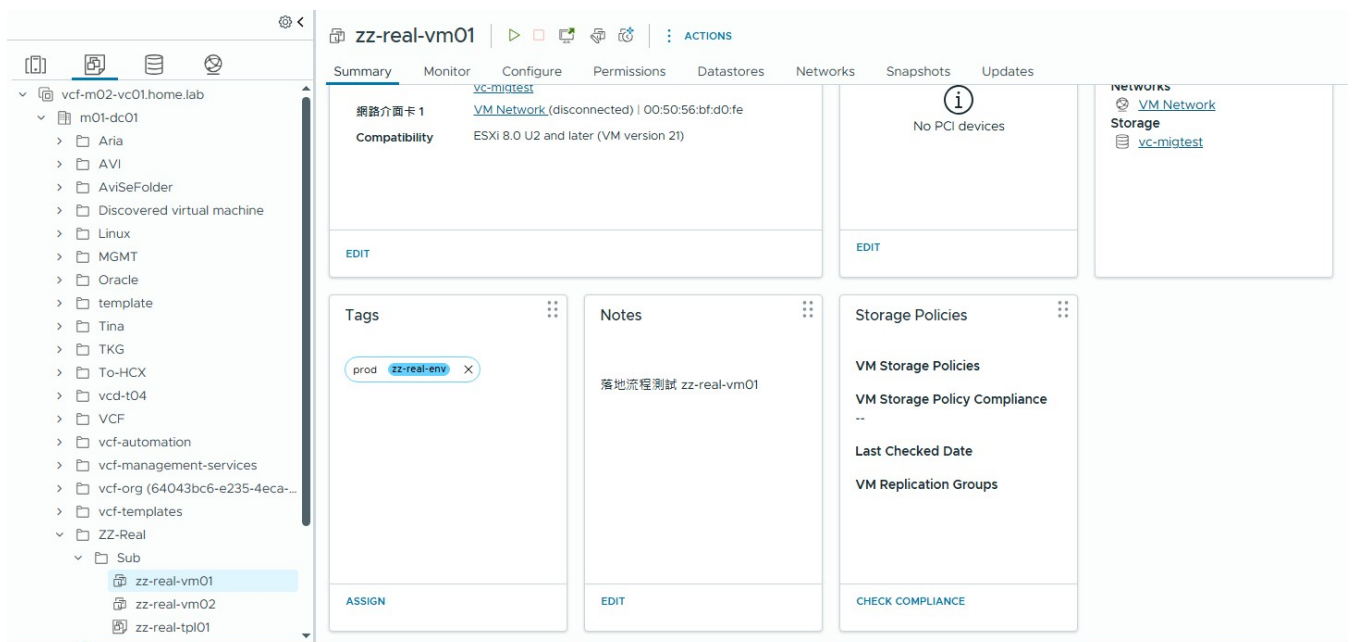


圖 9 新 vC：動 5 之後 zz-real-vm01 的 Tags 已是 prod / zz-real-env

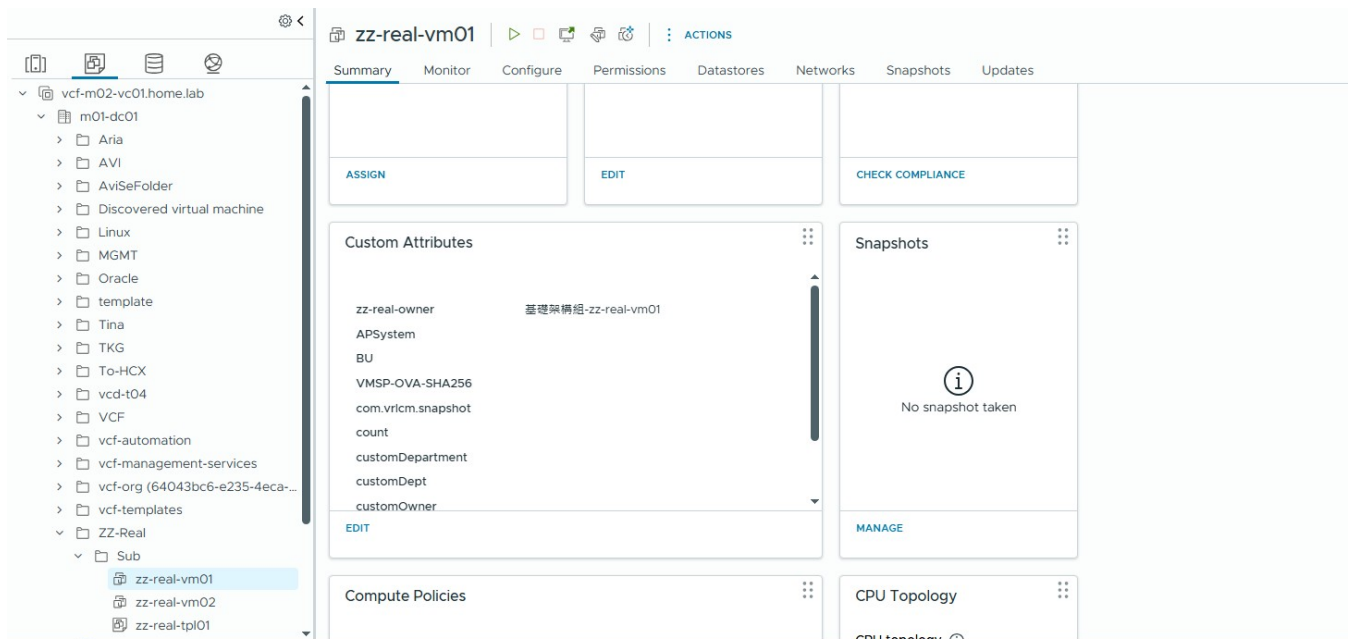


圖 10 新 vC：Custom Attributes 卡片 zz-real-owner = 基礎架構組-zz-real-vm01，與舊 vC 一致

7.8 核對結果

用獨立查詢（不靠腳本自己的回報）從新 vC 抓兩台 VM 的實際狀態，與舊 vC 匯出時比對：

項目	zz-real-vm01	zz-real-vm02	與舊 vC
Datcenter / 資料夾	m01-dc01 / ZZ-Real/Sub	m01-dc01 / ZZ-Real/Sub	一致
vmx	[vc-migtest] zz-real-vm01/zz-real-vm01.vmx	[vc-migtest] zz-real-vm02/zz-real-vm02.vmx	一致
自訂屬性 zz-real-owner	基礎架構組-zz-real-vm01	基礎架構組-zz-real-vm02	一致（中文）
Notes	落地流程測試 zz-real-vm01	落地流程測試 zz-real-vm02	一致
Tag	zz-real-env/prod	zz-real-env/prod	一致
InstanceUuid	503f0e10-ee95-2496-4e63-5372b4a22aa6	503f4f6e-1f0a-ee9a-5e4a-36cd766d8635	一致（跨 vC 保留）
電源	PoweredOff	PoweredOff	未被開機
舊 vC inventory	—	—	已無 zz-real-vm*

結論：五個動作照順序各跑一次即完成搬移；資料夾、自訂屬性值、Notes、tag 指派與舊 vC 完全一致，InstanceUuid 跨 vC 保留（因此動 5 的 UUID 比對在真實搬移也有效）。測試後 VM、資料夾、屬性、tag、兩邊 datastore 掛載與 NFS export 全部拆除，兩邊複驗無殘留。

7.9 冪等性與分批

- 動 4 分兩批跑（先 vm01、後 vm02），第二批對 vm01 回報 AlreadyRegistered，對帳從「還沒過來 1 台」變 0 台。
- 動 5 在 vm02 補回來後再跑一次：vm01 的值 / 指派回報 AlreadySet / Exists，vm02 回報 Set / Assigned——只補缺的。
- 動 2 過程中新 vC 的 tagging 服務曾回 503，重跑同一動即過，前一次已建好的屬性定義回報 Exists，沒有重做。

7.10 自我測試

兩支自我測試腳本把「建測試物件 → 跑 → 獨立驗證 → 清除」包起來，任何一項失敗 exit 1，可掛排程定期驗證環境：

腳本	涵蓋	結果
Test-VcMeta.ps1	匯出 4、範圍匯出 4、DryRun 2、範圍匯入 2、匯入 2、目標端獨立驗證 5、清除複驗 2	21 / 21 PASS
Test-RegisterVmx.ps1	建 VM+範本（資料夾 / 中文屬性 / Notes / tag）→ 匯出快照 → unregister 並刪資料夾 / 屬性 / tag → 註冊 → 逐項驗證 → 冪等	19 / 19 PASS

腳本	涵蓋	結果
	→ 清除	

8. 附錄：實作過程抓到的坑

這些都是實跑才冒出來、單機模擬測不到的，已在腳本裡處理：

坑	現象	處理
RegisterVM 的 name 不能空	API 說 name 省略就用 vmx 的 displayName，但 PowerCLI 把 \$null 送成 "，vCenter 回 "is invalid	腳本自己抓 vmx 讀 displayName (VM 改過名但沒 storage vMotion 時，檔名 ≠ 實際名稱)，讀不到才用檔名
PowerShell -like 的中括號	-like "[ds] *" 把 [ds] 當字元集合 wildcard，datastore 篩選全部落空	改用 String.StartsWith
陣列 splatting 只做位置綁定	呼叫子腳本 mandatory 參數沒綁到，pwsh 停在互動 prompt 等輸入	一律 hashtable splatting
unregister 後舊 vC 就沒資料	VM 一離開 inventory，tag 指派與屬性值消失，現場抓抓不到	動 0 必須在動 3 之前；動 3 沒匯出檔不給做
範本 (Template)	Get-VM -Id 抓不到、Set-VM -Notes / Move-VM 不能用	MoRef 轉物件 + ReconfigVM / Move-Template
來源有掛掉的 datastore	Get-TagAssignment 一次撈整批中斷	分批撈，失敗再逐一 fallback
tagging 服務 503	vAPI/CIS 在 session 剛建立時偶爾 Service Unavailable	tag 相關呼叫 3 次退避重試

原始碼與 README：<https://github.com/kostenyang/vc-folder-tag-migrate>